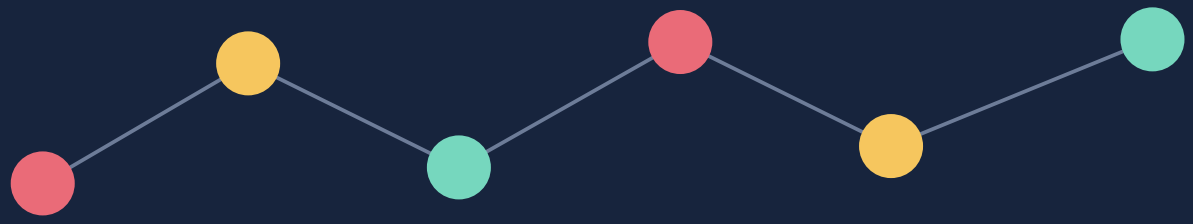
GATE 2027 / COMPLETE SYLLABUS EDITION

Compiler Design

Start from the basics. Build every syllabus idea. Then practise.

Section 7: Compiler Design



LEARN → SEE → WORK THROUGH → RETRIEVE → PRACTISE

1 Understand

Each chapter starts in plain language.

2 Visualize

Follow the flow and redraw it.

3 Work

Trace the worked example with pencil.

4 Recall

Answer before checking the solution.

A beginner learning guide + illustrated recall/practice + your supplied reference material

2027 syllabus coverage map

Use this as a route map: every item below has a beginner lesson in this PDF. The supplied handbook/chapter PDFs are gathered in the reference section after the learning and practice pages.

- [ ] **Compiler pipeline and lexical analysis**

  Lexical analysis • Patterns and state • Symbol table • Errors

- [ ] **Parsing and grammars**

  Context-free grammar • Top-down parsing • Bottom-up parsing • Ambiguity and precedence

- [ ] **Syntax-directed translation**

  Attributes • Type checking • Intermediate actions • Dependency order

- [ ] **Runtime environments**

  Activation record • Scope and lifetime • Parameter passing • Heap and stack

- [ ] **Intermediate code and local optimization**

  IR • Basic block • Local transformations • Code generation

- [ ] **Data-flow analysis: constants, liveness and common subexpressions**

  Constant propagation • Liveness • Equations • Common subexpression elimination

- [ ] **Data-flow analysis: GEN/KILL and fixed points**

  Reaching definitions • Available expressions • Liveness • Fixed point

- [ ] **Attribute dependencies, backpatching and basic blocks**

  Dependency order • Basic block leaders • Backpatch lists • Local optimization

Your route through this subject

Follow the lessons in order. Tick a topic only after you can explain its picture without looking; a tick means “I can teach it,” not “I have seen it.”

1. **Compiler pipeline and lexical analysis**
2. **Parsing and grammars**
3. **Syntax-directed translation**
4. **Runtime environments**
5. **Intermediate code and local optimization**
6. **Data-flow analysis: constants, liveness and common subexpressions**
7. **Data-flow analysis: GEN/KILL and fixed points**
8. **Attribute dependencies, backpatching and basic blocks**

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

IF THIS SUBJECT FEELS COMPLETELY NEW

That is okay. Start with the first lesson, keep a pencil beside you, and redraw its picture once. Attempt the recall prompt before opening the answer guide.

Compiler pipeline and lexical analysis

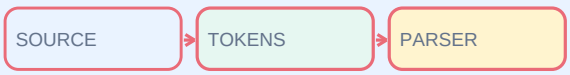
START WITH THE IDEA

A compiler translates a source program into a target form while checking structure and preserving meaning. Each stage consumes a representation and adds information.

A CONCRETE FIRST EXAMPLE

Input `count = a+12;` becomes tokens IDENT, ASSIGN, IDENT, PLUS, NUMBER, SEMICOLON.

PICTURE IT



IMAGINE IT THIS WAY

The lexer is a word separator; the parser is the sentence checker.

THE IDEAS, ONE BY ONE

Lexical analysis

The lexer groups characters into tokens such as identifiers, numbers and operators; whitespace/comments are usually skipped.

Patterns and state

Token classes are commonly described by regular expressions and implemented by finite automata.

Symbol table

Stores identifier attributes such as name, scope, type and location.

Errors

A lexical error is an invalid token; later parser errors concern token arrangement.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT A token's lexeme is its actual text; its token class is its category.

TRY FROM MEMORY

Cover this page. Explain Compiler pipeline and lexical analysis using the picture, then give one example before looking again.

Parsing and grammars

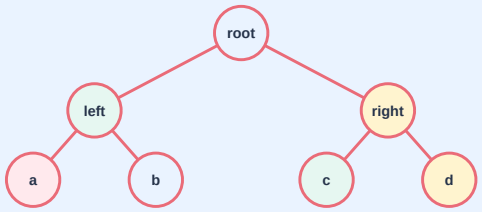
START WITH THE IDEA

Parsing checks whether the token sequence fits the language grammar and builds a tree that exposes syntactic structure.

A CONCRETE FIRST EXAMPLE

For `a+b*c`, precedence makes the parse tree `a+(b*c)` rather than `(a+b)*c`.

PICTURE IT



IMAGINE IT THIS WAY

A parse tree is the sentence diagram of a program.

THE IDEAS, ONE BY ONE

Context-free grammar

Productions define legal expansions for expressions, statements and declarations.

Top-down parsing

Predictive/LL methods build from the start symbol and choose productions using lookahead; left recursion is a problem.

Bottom-up parsing

Shift-reduce/LR methods build from input pieces to the start symbol; they use handles and parse tables.

Ambiguity and precedence

An ambiguous grammar can give one input multiple parse trees. Precedence/associativity rules determine intended grouping.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT Syntax validity does not guarantee type or runtime correctness.

TRY FROM MEMORY

Cover this page. Explain Parsing and grammars using the picture, then give one example before looking again.

Syntax-directed translation

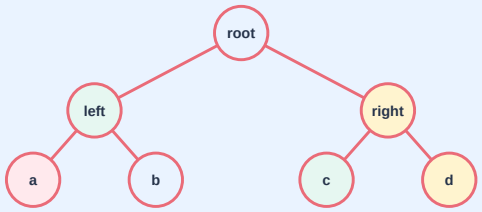
START WITH THE IDEA

Translation rules attached to grammar productions move semantic information through the parse tree and can generate code or check meaning.

A CONCRETE FIRST EXAMPLE

For E -> E_1+T, a synthesized type rule checks numeric operands and sets type(E) to their common numeric type.

PICTURE IT



IMAGINE IT THIS WAY

Think of every parse-tree node carrying a small labeled note to its parent or child.

THE IDEAS, ONE BY ONE

Attributes

Synthesized attributes flow from children upward; inherited attributes flow from parent or siblings downward/across.

Type checking

Rules ensure operators receive compatible operand types and expressions produce valid types.

Intermediate actions

Actions may build syntax trees, symbol-table entries or intermediate instructions.

Dependency order

An attribute can be evaluated only after the attributes it depends on are available.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT Keep inherited and synthesized attributes distinct.

TRY FROM MEMORY

Cover this page. Explain Syntax-directed translation using the picture, then give one example before looking again.

Runtime environments

START WITH THE IDEA

Generated programs need a memory layout for variables, parameters, return addresses and dynamic objects. Calling conventions coordinate caller and callee.

A CONCRETE FIRST EXAMPLE

During nested calls, each activation record is pushed; return restores the saved state and resumes at the caller's return address.

PICTURE IT



IMAGINE IT THIS WAY

A stack frame is a call's temporary desk; the heap is a shared storage area for dynamic objects.

THE IDEAS, ONE BY ONE

Activation record

A stack frame may hold parameters, local variables, saved registers, control link and return address.

Scope and lifetime

Static scope determines which declaration a name refers to; lifetime determines how long its storage exists.

Parameter passing

Value, reference or address conventions change whether updates in a function affect caller-visible data.

Heap and stack

Stack supports nested calls; heap supports dynamic objects whose lifetime may outlast a call.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT Scope and lifetime are different: where a name is visible versus how long storage exists.

TRY FROM MEMORY

Cover this page. Explain Runtime environments using the picture, then give one example before looking again.

Intermediate code and local optimization

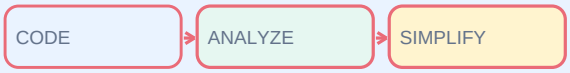
START WITH THE IDEA

Intermediate representation (IR) makes translation easier by separating source-language details from target-machine details. Local optimization improves a small region without changing behavior.

A CONCRETE FIRST EXAMPLE

Replace `t1=3*4; t2=a+t1` with `t2=a+12` by constant folding then propagation.

PICTURE IT



IMAGINE IT THIS WAY

IR is a shared translation language between source meaning and machine instructions.

THE IDEAS, ONE BY ONE

IR

Three-address code uses simple operations like `t1=a*b`; syntax trees and SSA are other representations.

Basic block

A maximal straight-line sequence with one entry and one exit; control flow is represented between blocks.

Local transformations

Constant folding, algebraic simplification, copy propagation and dead-code removal simplify a block.

Code generation

Instruction selection, register allocation and scheduling map IR to target operations.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT Optimization must preserve observable behavior, including side effects.

TRY FROM MEMORY

Cover this page. Explain Intermediate code and local optimization using the picture, then give one example before looking again.

Data-flow analysis: constants, liveness and common subexpressions

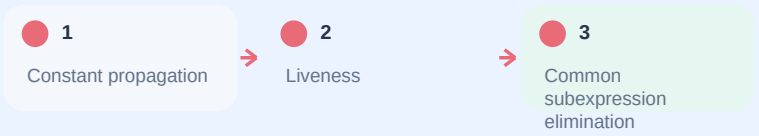
START WITH THE IDEA

Data-flow facts flow through a control-flow graph. Iteration reaches a fixed point that tells the compiler which values are known or still needed.

A CONCRETE FIRST EXAMPLE

If both branches assign x=5, x is constant after the merge. If one assigns 5 and one 7, it is not.

PICTURE IT



IMAGINE IT THIS WAY

Data-flow analysis passes sticky notes along control-flow arrows until no note changes.

THE IDEAS, ONE BY ONE

Constant propagation

Track values that stay constant along all incoming paths. At a merge, keep only facts true on every path.

Liveness

A variable is live if its current value may be used later before being overwritten.

Equations

OUT[B]=union IN[successors]; IN[B]=USE[B] union (OUT[B]-DEF[B]) for standard liveness.

Common subexpression elimination

Reuse a computed expression only if its operands have not changed and it is safe to preserve effects.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT At a join, do not assume a fact that is true on only one incoming path.

TRY FROM MEMORY

Cover this page. Explain Data-flow analysis: constants, liveness and common subexpressions using the picture, then give one example before looking again.

Data-flow analysis: GEN/KILL and fixed points

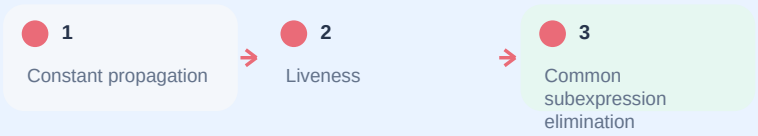
START WITH THE IDEA

A compiler can ask what facts are true at each program point. Each basic block generates facts and kills facts; repeated propagation reaches a fixed point on loops.

A CONCRETE FIRST EXAMPLE

If a block assigns x, it kills old definitions of x and generates the new definition. For liveness, a use before a definition belongs in USE.

PICTURE IT



IMAGINE IT THIS WAY

Facts are sticky notes moving through a flowchart. A fixed point means another lap changes none of them.

THE IDEAS, ONE BY ONE

Reaching definitions

Forward may analysis: IN[B]=union OUT[pred]; OUT[B]=GEN[B] union (IN[B] minus KILL[B]).

Available expressions

Forward must analysis: IN[B]=intersection OUT[pred]; OUT[B]=GEN[B] union (IN[B] minus KILL[B]). Initialize non-entry facts to the universal set.

Liveness

Backward may analysis: OUT[B]=union IN[succ]; IN[B]=USE[B] union (OUT[B] minus DEF[B]).

Fixed point

Initialize boundary facts, process blocks until no IN or OUT set changes. Loops require iteration, not one pass.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT Use predecessor unions for forward may facts, predecessor intersections for forward must facts, and reverse edges for backward facts.

TRY FROM MEMORY

Cover this page. Explain Data-flow analysis: GEN/KILL and fixed points using the picture, then give one example before looking again.

Attribute dependencies, backpatching and basic blocks

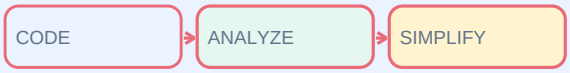
START WITH THE IDEA

Syntax-directed translation attaches meaning or code to grammar rules. When a jump target is not known yet, backpatching fills it in after the target is created.

A CONCRETE FIRST EXAMPLE

For if x<y goto _; t=x+y; goto _; true: ... , backpatch the conditional target to true and the unconditional target to the join point.

PICTURE IT



IMAGINE IT THIS WAY

Backpatching is leaving a blank destination label on a jump and filling it in when the destination exists.

THE IDEAS, ONE BY ONE

Dependency order

Draw an edge from each attribute to the attribute it needs. Evaluate a topological order; a cycle means this dependency graph has no simple evaluation order.

Basic block leaders

Leaders are the first instruction, every jump target, and every instruction immediately after a jump.

Backpatch lists

Maintain true-list and false-list for Boolean tests and next-list for fall-through jumps. Patch each list when its destination becomes known.

Local optimization

Within a basic block, apply constant folding, copy propagation, common-subexpression elimination and dead-code removal while preserving side effects.

HOW TO USE IT ON A PROBLEM

Identify the compiler stage and current representation. Apply one grammar or semantic rule at a time, show the tree/IR result, and track names and types.

WATCH OUT Never move an expression across a definition of one of its operands.

TRY FROM MEMORY

Cover this page. Explain Attribute dependencies, backpatching and basic blocks using the picture, then give one example before looking again.

Past-paper trail and coverage check

WHAT THE PAPERS MATCH

Verified anchors: 2018 Q37-Q38 (lexical analysis and parsing); 2022 Q13/Q29/Q65 (LALR/LR and syntax-directed translation); 2023 Q18/Q59 (lexical tokens and translation); 2024 CS1 Q26/Q37/Q39; 2025 CS1 Q12-Q13/Q46/Q52 and CS2 Q21-Q22/Q40-Q41/Q51; 2026 CS1 Q27-Q28/Q42/Q53 and CS2 Q35/Q41/Q45.

WHAT THIS EDITION ADDS

Added precise GEN/KILL equations for reaching definitions, available expressions and liveness, fixed-point iteration, dependency ordering, leaders and backpatching.

SYLLABUS BOUNDARY

The user-supplied Gemini answer described several 2025 CD question types incorrectly; the listed verified anchors match the actual session PDFs.

HOW TO USE THIS CHECK

A question is a match when its tested skill is taught in the lesson pages, even if the story or numbers are new. Anchors are selected verified examples, not an exhaustive list; the separate Coverage and Gap Audit PDF identifies years whose scans could not be checked question by question.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

1. **Compiler pipeline and lexical analysis**

   Without notes: define Errors and explain why this example works. Then recreate the steps on the lesson page.

2. **Parsing and grammars**

   Without notes: define Ambiguity and precedence and explain why this example works. Then recreate the steps on the lesson page.

3. **Syntax-directed translation**

   Without notes: define Dependency order and explain why this example works. Then recreate the steps on the lesson page.

4. **Runtime environments**

   Without notes: define Heap and stack and explain why this example works. Then recreate the steps on the lesson page.

5. **Intermediate code and local optimization**

   Without notes: define Code generation and explain why this example works. Then recreate the steps on the lesson page.

6. **Data-flow analysis: constants, liveness and common subexpressions**

   Without notes: define Common subexpression elimination and explain why this example works. Then recreate the steps on the lesson page.

Close the notes: try before you peek

For each card, explain the idea from memory and recreate the lesson example on paper.

7 **Data-flow analysis: GEN/KILL and fixed points**

Without notes: define Fixed point and explain why this example works. Then recreate the steps on the lesson page.

8 **Attribute dependencies, backpatching and basic blocks**

Without notes: define Local optimization and explain why this example works. Then recreate the steps on the lesson page.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

1. **LESSON 1: Compiler pipeline and lexical analysis**

   Input `count = a+12;` becomes tokens IDENT, ASSIGN, IDENT, PLUS, NUMBER, SEMICOLON.

2. **LESSON 2: Parsing and grammars**

   For `a+b*c`, precedence makes the parse tree `a+(b*c)` rather than `(a+b)*c`.

3. **LESSON 3: Syntax-directed translation**

   For E -> E_1+T, a synthesized type rule checks numeric operands and sets type(E) to their common numeric type.

4. **LESSON 4: Runtime environments**

   During nested calls, each activation record is pushed; return restores the saved state and resumes at the caller’s return address.

5. **LESSON 5: Intermediate code and local optimization**

   Replace `t1=3*4; t2=a+t1` with `t2=a+12` by constant folding then propagation.

6. **LESSON 6: Data-flow analysis: constants, liveness and common subexpressions**

   If both branches assign x=5, x is constant after the merge. If one assigns 5 and one 7, it is not.

Compare with the worked examples

Use this only after attempting the recall prompts. The full reasoning is on the numbered lesson page.

7. LESSON 7: Data-flow analysis: GEN/KILL and fixed points

If a block assigns x, it kills old definitions of x and generates the new definition. For liveness, a use before a definition belongs in USE.

8. LESSON 8: Attribute dependencies, backpatching and basic blocks

For if x<y goto _; t=x+y; goto _; true: ... , backpatch the conditional target to true and the unconditional target to the join point.

Your practice and reference library

01 **Beginner lessons**

Plain-language explanation, visual sequence, example and watch-outs for every listed 2027 topic cluster.

02 **Recall and GATE-style practice**

The illustrated recap section includes questions and a separate answer explanation.

03 **Detailed reference**

Your supplied subject handbook and chapter PDFs follow for extra depth, reading and worked material.

Included source documents

- **Illustrated recall and GATE-style practice** — CD - Compiler Design - GATE 2027 Illustrated Notes.pdf — 11 pages
- **Detailed supplied reference** — CD.pdf — 26 pages

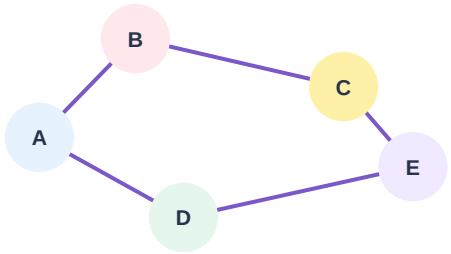
The official 2027 syllabus map precedes the lessons. Original reference pages are retained so you can use them for slower reading and extra practice.

Compiler Design

GATE 2027 | VISUAL LEARNING NOTES | STARTING FROM THE BASICS

Follow a program from characters to meaning to efficient code.

PICTURE IT



characters -> tokens

HOW TO USE THIS NOTEBOOK

1. **Learn the idea**
   Read the plain-language explanation before memorizing a formula.
2. **See it**
   Study the labeled picture and redraw it from memory.
3. **Follow a solve**
   Work through the examples step by step.
4. **Test yourself**
   Attempt the GATE-style questions before opening the key.

THE PATH YOU WILL FOLLOW

Front end	Meaning and representation	Optimization and data flow

IF YOU FEEL COMPLETELY NEW

That is okay. Start with the first topic page, keep a pencil beside you, and copy each diagram once. Do not jump straight to the answer key. The questions are meant to come after the concepts.

Your route through this subject

Follow the topics in order. Each page teaches one idea, shows a picture, gives a recall cue and points out a common trap.

FRONT END

- [ ] Lexical analysis.
- [ ] Parsing.
- [ ] Syntax-directed translation.

MEANING AND REPRESENTATION

- [ ] Runtime environments.
- [ ] Intermediate code generation.

OPTIMIZATION AND DATA FLOW

- [ ] Local optimization.
- [ ] Data-flow analyses: constant propagation, liveness analysis and common-subexpression elimination.

THE 20-MINUTE LEARNING LOOP

5 min	5 min	7 min	3 min
Read	Redraw	Solve	Recall

Tick a topic only after you can explain its picture without looking. A tick means "I can teach it," not "I have seen it."

BEFORE EACH STUDY SESSION

Pick one topic. Read its idea, redraw its diagram, follow the worked examples, then answer questions with the notes closed. Circle only the step where you got stuck - that is your next revision target.

CONCEPT 01 / CD

Lexical analysis

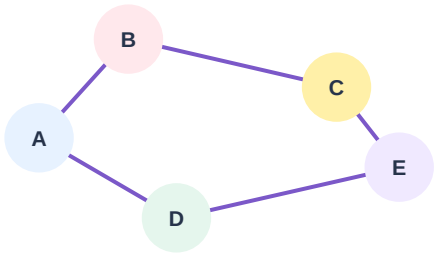
START WITH THE IDEA

The lexer groups characters into tokens using patterns; whitespace/comments are usually discarded. Longest-match and priority rules resolve overlapping token patterns. A symbol table records identifiers and attributes.

A CONCRETE FIRST EXAMPLE

A lexer groups characters into tokens such as identifiers, numbers and operators. It usually follows longest-match and priority rules; whitespace and comments may be discarded while line positions are kept. For "count=12", tokens are ID(count), ASSIGN(=), NUM(12). The lexer does not decide whether count was declared.

PICTURE IT



characters -> tokens

IMAGINE IT THIS WAY

The lexer is a mail sorter: characters become labeled packages such as ID, NUM and PLUS.

HOW TO USE IT ON A PROBLEM

Trace characters left to right and mark token boundaries.

WATCH OUT

A lexer recognizes token shape, not the full grammatical structure.

TRY FROM MEMORY

Cover the page and explain Lexical analysis using the diagram. Give one example before looking again.

CONCEPT 02 / CD

Parsing and grammar

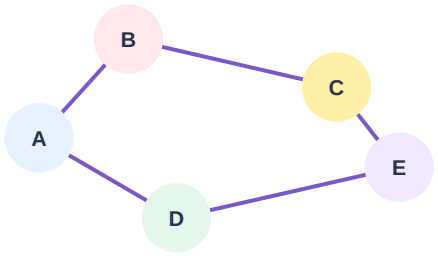
START WITH THE IDEA

A parser checks whether tokens follow a grammar. Top-down methods build from the start symbol; bottom-up methods reduce handles. FIRST/FOLLOW sets support predictive parsing; conflicts can signal ambiguity or grammar limitations.

A CONCRETE FIRST EXAMPLE

A parser checks whether the token sequence follows a grammar and builds structure. Ambiguity means one string has multiple parse trees; left recursion can be a problem for top-down parsers. For a+b*c, precedence rules should build a tree with + at the root and * under the right child.

PICTURE IT



tokens -> syntax tree

IMAGINE IT THIS WAY

Parsing is assembling a sentence using a grammar blueprint.

HOW TO USE IT ON A PROBLEM

For LL(1), alternatives need disjoint FIRST sets, with FOLLOW considered for epsilon.

WATCH OUT

Left recursion breaks naive recursive-descent parsing.

TRY FROM MEMORY

Cover the page and explain Parsing and grammar using the diagram. Give one example before looking again.

CONCEPT 03 / CD

Syntax-directed translation

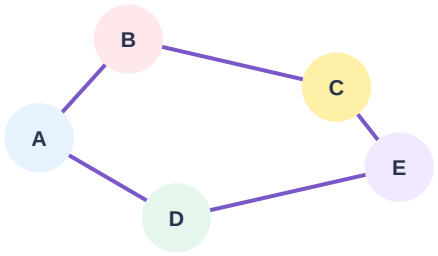
START WITH THE IDEA

Attributes carry semantic information through a parse tree. Syntax-directed definitions attach rules to productions; translation actions can build trees or emit code.

A CONCRETE FIRST EXAMPLE

Syntax-directed translation attaches meaning or actions to grammar rules. It can compute types, values or intermediate code while the parser builds the syntax tree. For E -> E1+T, an action can emit code that adds the locations holding E1 and T.

PICTURE IT



tree -> intermediate code

IMAGINE IT THIS WAY

An annotated parse tree is a recipe with measurement notes at each branch.

HOW TO USE IT ON A PROBLEM

Keep inherited (from context) and synthesized (from children) attributes distinct.

WATCH OUT

Syntactic validity does not guarantee type or scope correctness.

TRY FROM MEMORY

Cover the page and explain Syntax-directed translation using the diagram. Give one example before looking again.

CONCEPT 04 / CD

Runtime environments

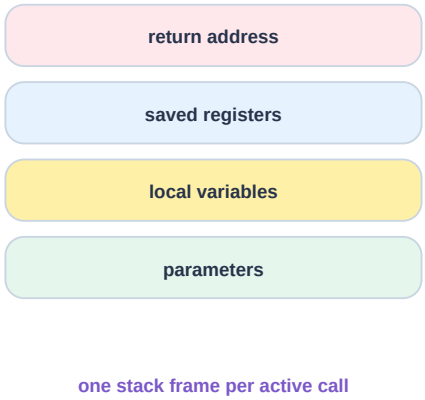
START WITH THE IDEA

Activation records store parameters, locals, return address and saved state. Stack allocation fits nested calls; static links or displays can access nonlocal variables; heap allocation supports dynamic lifetime.

A CONCRETE FIRST EXAMPLE

A procedure call needs a runtime activation record for parameters, local variables, saved registers and return address. Recursion needs a separate record per active call. While f(2) calls f(1), both records exist; returning from f(1) resumes the saved return point in f(2).

PICTURE IT



IMAGINE IT THIS WAY

Each function call gets a desk drawer; nested calls open new drawers until return.

HOW TO USE IT ON A PROBLEM

Draw the call stack and lexical nesting for scope questions.

WATCH OUT

Lifetime and scope are different: a variable can be in scope but not live after its block.

TRY FROM MEMORY

Cover the page and explain Runtime environments using the diagram. Give one example before looking again.

CONCEPT 05 / CD

Intermediate code and local optimization

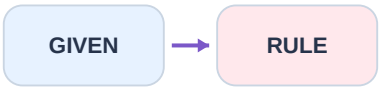
START WITH THE IDEA

Three-address code breaks expressions into simple operations. Basic blocks have one entry and one exit. Local optimizations include constant folding/propagation, copy propagation, dead-code removal and common-subexpression elimination.

A CONCRETE FIRST EXAMPLE

Intermediate code makes optimization independent of the target machine. Local optimizations include constant folding, common-subexpression elimination and dead-code removal, while preserving program behavior. Replace 3*4 with 12 at compile time; if x is never used after x=y+1, that assignment may be dead.

PICTURE IT



IMAGINE IT THIS WAY

IR is a common language between front ends and machine back ends.

HOW TO USE IT ON A PROBLEM

Track definitions and uses before deleting or reusing a computation.

WATCH OUT

A repeated expression can be reused only when its operands have not changed.

TRY FROM MEMORY

Cover the page and explain Intermediate code and local optimization using the diagram. Give one example before looking again.

CONCEPT 06 / CD

Data-flow analysis: liveness and constants

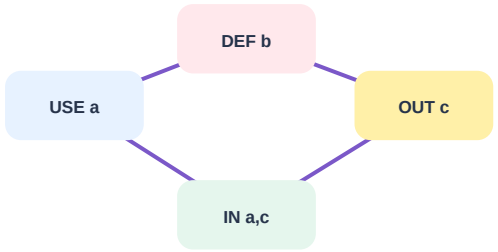
START WITH THE IDEA

Liveness equations: OUT[B]=union IN[successors]; IN[B]=USE[B] union (OUT[B]-DEF[B]). Constant propagation solves value facts across control-flow joins; a join keeps only facts true on every incoming path.

A CONCRETE FIRST EXAMPLE

Data-flow analysis repeatedly propagates facts through a control-flow graph until they stop changing. Liveness asks whether a variable's current value may be used later; constants analysis tracks known values. For a node, IN=USE union (OUT-DEF). If USE={a,c}, DEF={b}, OUT={b,c,d}, then IN={a,c,d}.

PICTURE IT



facts flow backward through the graph

IMAGINE IT THIS WAY

Data-flow facts are sticky notes passed along control-flow arrows until a fixed point.

HOW TO USE IT ON A PROBLEM

Initialize, transfer, join, iterate until no set changes.

WATCH OUT

At a branch merge, a variable is not constant if incoming paths disagree.

TRY FROM MEMORY

Cover the page and explain Data-flow analysis: liveness and constants using the diagram. Give one example before looking again.

Let's solve it together

Cover the answer with your hand. Try each step first, then compare your reasoning.

EXAMPLE 1: Liveness equation

QUESTION

A block uses x, defines y, and has OUT={x,z}. Compute IN.

WORK THROUGH THE STEPS

USE={x}; DEF={y}. IN=USE union

ANSWER

IN={x,z}; y is not live-in unless used before definition.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

EXAMPLE 2: Constant propagation at a join

QUESTION

Path 1 reaches block B with x=4; path 2 reaches B with x=4. Is x constant at B? What if path 2 has x=5?

WORK THROUGH THE STEPS

At a join, retain a constant only if every incoming path agrees. Both 4 -> x=4. Values 4 and 5 -> x is

ANSWER

First case: x=4. Second: unknown/nonconstant.

Check: write the key rule beside your answer. If it does not fit the rule, redo the model before recalculating.

Now you try

Attempt these without the notes. The question types are marked MCQ, MSQ or NAT. Answers and explanations are on the next page.

MCQ Which compiler phase normally groups characters into identifiers, numbers and operators?

(A) Parser (B) Lexer (C) Optimizer (D) Register allocator

MCQ For predictive parsing, left recursion is problematic because it can:

(A) Cause infinite recursion (B) Remove all terminals (C) Make every grammar regular (D) Eliminate ambiguity automatically

NAT A block has USE={a,c}, DEF={b}, OUT={b,c,d}. Compute |IN| using IN=USE union (OUT-DEF).

Enter the set size.

MSQ Which are local optimization examples?

(A) Constant folding (B) Common-subexpression elimination (C) Liveness analysis (D) Copy propagation

MCQ At a control-flow join, x is constant only if:

(A) One path defines it (B) All incoming paths agree on its value (C) It is live (D) It appears in the symbol table

Exam habit: decide what is being asked before doing arithmetic. Show units, assumptions and one line of reasoning.

Answers with the why

Read the explanation even when your answer is correct. The goal is to learn the reasoning pattern.

1. B

WHY

Lexical analysis produces the token stream.

2. A

WHY

A recursive-descent call can re-enter the same nonterminal without consuming input.

3. 3

WHY

OUT-DEF={c,d}; union USE={a,c,d}, size 3.

4. A, B, D

WHY

Liveness is a data-flow analysis that can enable optimizations; it is not itself one of the listed local rewrites.

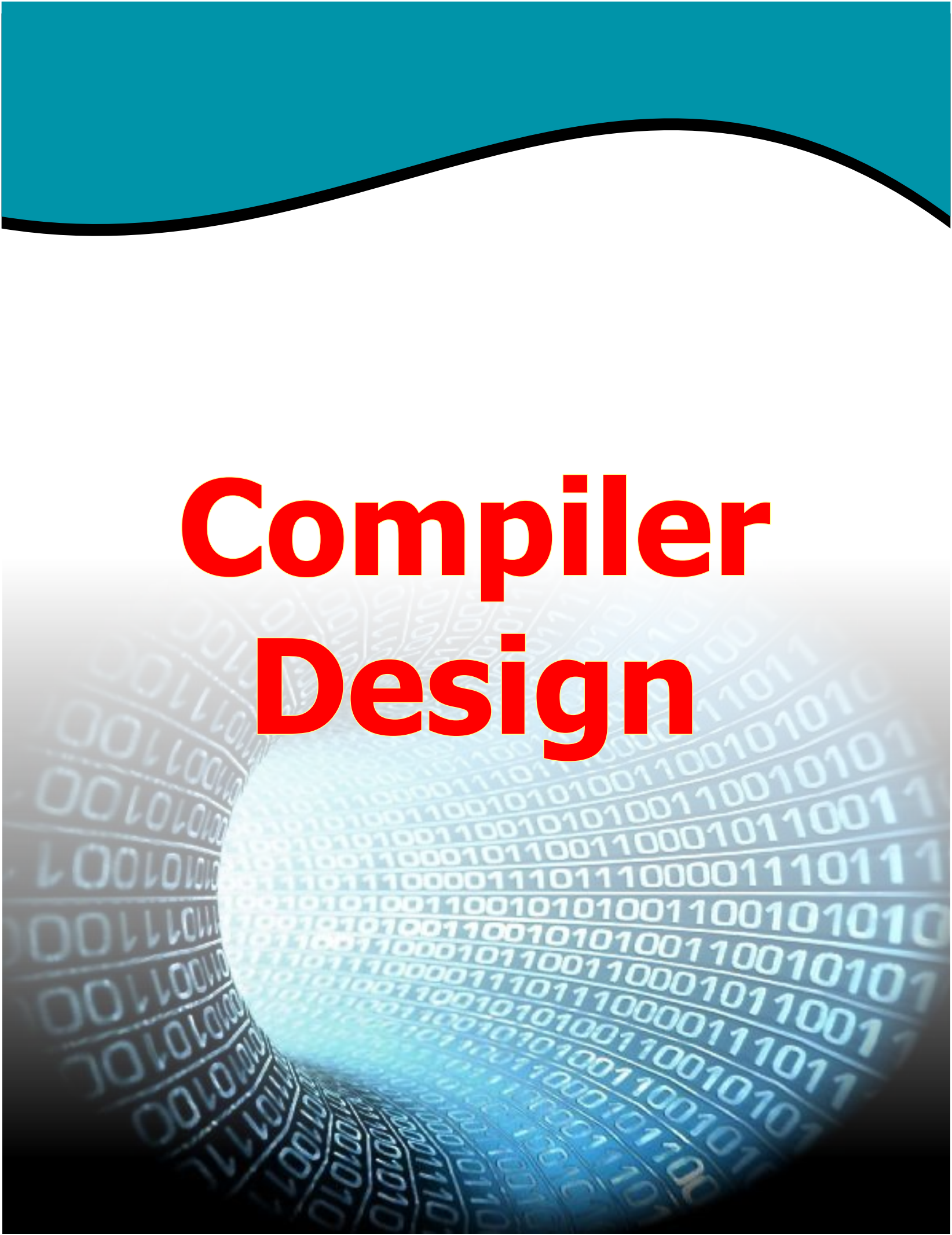
5. B

WHY

A constant fact must hold on every path reaching the point.

Review loop: mark each missed concept page, redraw its picture, and retry a similar problem tomorrow.

Compiler Design



Published By:

Physics Wallah

ISBN: 978-93-94342-39-2

Mobile App: Physics Wallah (Available on Play Store)

Website: www.pw.live

Email: support@pw.live

Rights

All rights will be reserved by Publisher. No part of this book may be used or reproduced in any manner whatsoever without the written permission from author or publisher.

In the interest of student's community:

Circulation of soft copy of Book(s) in PDF or other equivalent format(s) through any social media channels, emails, etc. or any other channels through mobiles, laptops or desktop is a criminal offence. Anybody circulating, downloading, storing, soft copy of the book on his device(s) is in breach of Copyright Act. Further Photocopying of this book or any of its material is also illegal. Do not download or forward in case you come across any such soft copy material.

Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

This book and the individual contribution contained in it are protected under copyright by the publisher.

(This Module shall only be Used for Educational Purpose.)

Compiler Design

INDEX

1. Introduction and Lexical Analysis ..... 8.1 – 8.2
2. Parsing ..... 8.3 – 8.11
3. Lexical Analysis ..... 8.12 – 8.13
4. Syntax Directed Transaction ..... 8.14 – 8.16
5. Intermediate, Code Optimization ..... 8.17 – 8.23

1 INTRODUCTION AND LEXICAL ANALYSIS

1.1 CD Introduction

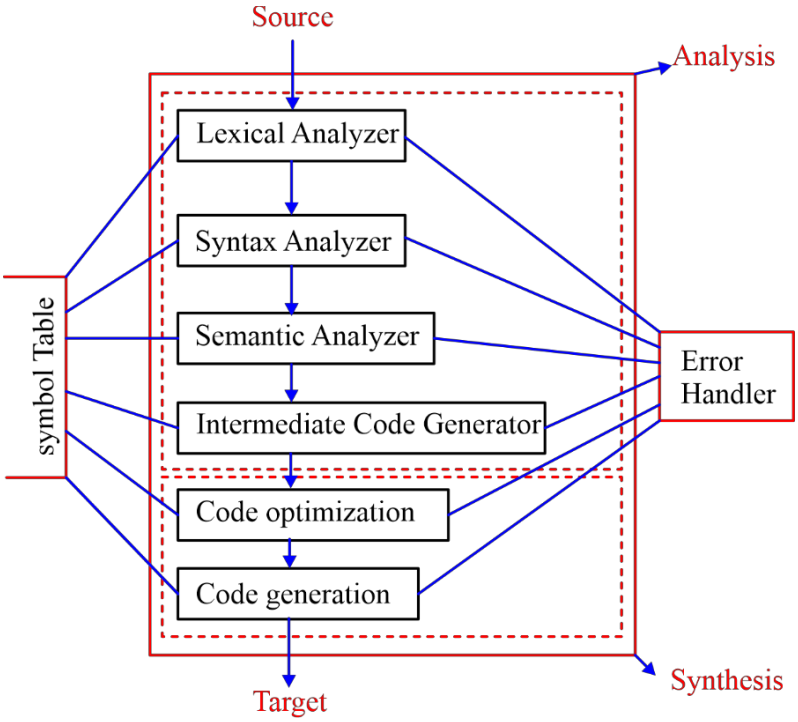
1.1.1 Definition

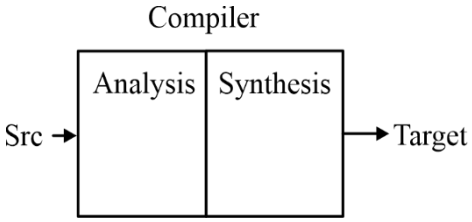
Convert High Level Language to Low Level Language.



- High Level Language can perform more than one operation in a single Statement.
- lol Level Language can perform atmost one operation in a statement.

1.2 Analysis and Synthesis model of Compiler





- There are 6 phases of the Compiler

1. **Lexical Analyzer:**

- Program of DFA, it checks for spelling mistakes of program.
- Divides source code into stream of tokens.

2. **Syntax Analyzer:**

- Checks grammatical errors of the program (Parser).
- Parser is a DPDA.

3. **Semantic Analyzer:**

Checks for meaning of the program.

Example:

Type miss match, stack overflow

4. **Intermediate Code Generation:**

- This phase makes the work of next 2 phases much easier.
- Enforces reusability and portability.

5. **Code optimization:**

- Loop invariant construct
- Common sub expression elimination
- Strength Reduction
- Function in lining

Deadlock elimination

6. **Symbol Table:**

(1) Data about Data (Meta data)

(2) Data structure used by compiler and shared by all the phrase.

❑❑❑

2 PARSING

2.1 CD - Grammar

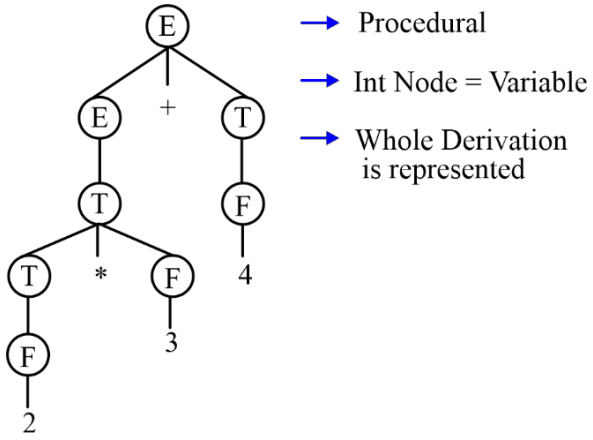
→ In compiler we only use: Type – 2 (CFG) and Type – 3 Regular grammar.
→ Compiler = Program of Grammar
→ Compiler = Membership algorithm
• Every programming Language is Context Sensitive Grammar (Context Sensitive Language)

2.1.1 Parse Tree and Syntax Tree

G: $E \to E + T / T$ $E \to E + T \to T + T \to T * F + T \to F * F + T \to 2 * F + T \to 2 * 3 + T \to 2 * 3 + F$
$T \to T * F / F$

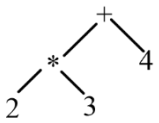
$E \to 2 * 3 + 4$

Parse Tree:



→ Procedural
→ Int Node = Variable
→ Whole Derivation is represented

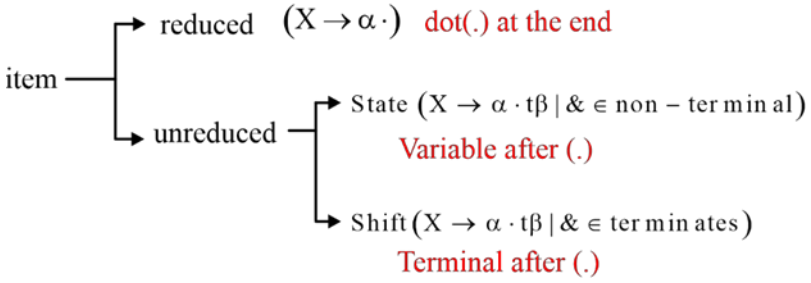
Syntax Tree:



→ non procedural
→ Int node = Operation
→ Discuss Concept only

- To check to priority / Associativity:
  Randomly derive till you have enough operators, then check which one is done first.
- If priority of 2 operators is same and both Left and Right associative → **Ambiguous Grammar** [Useless]

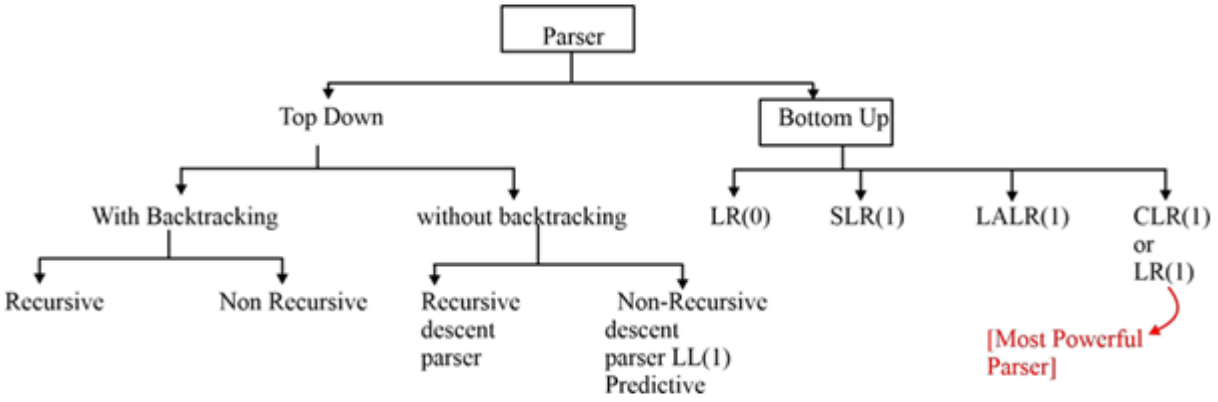
2.1.2 Types of Parsers in Bottom Up: [CD - Parser]

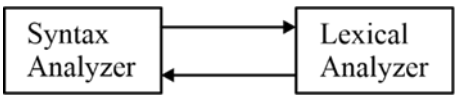


2.2 CD-Syntax Analysis / Parsing

Grammatical Errors are checked by the help of parsers.

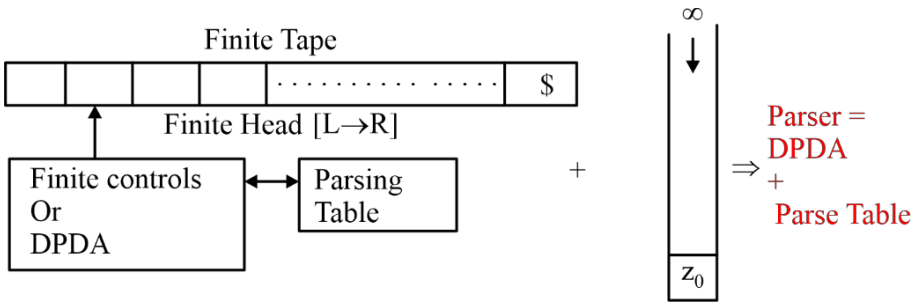
- Parsers are basically DPDA





- All of these parsers are table driven.

2.2.1 Mathematical Model of Parser



- Parsers generate Parse Tree, for a given string by the given grammar.

2.2.2 Top-Down Parser (LL (1))

- It uses LMD and is equivalent to DFS in graph.

2.2.3 Algorithm to construct Parsing Table

1. Remove Left Recursion if any.
2. Left Factor [Remove Common Prefix]
3. Find first and follow set
4. Construct the table.

- If we increase the look ahead symbol:

2.2.4 Removal of common Prefix: (Left Factor)

1. $S \rightarrow a \mid a\ b \mid a\ A\ S \rightarrow a\ Y$

   $Y \rightarrow \in \mid b \mid A$

2. $A \rightarrow a\ b\ A \mid a\ A \mid b$

   $A \rightarrow a \times \mid b \quad A \rightarrow a \times \mid b$

   $X \rightarrow b\ A \mid A \quad X \rightarrow b\ A \mid a\ x \mid b$

   $A \rightarrow a\ X \mid b$

   $X \rightarrow a\ X \mid b$

   $Y\ Y \rightarrow A \mid \in$

2.2.5 First and Follow

- First Set $\rightarrow$ extreme Left terminal from which the string of that variable starts.

  $\rightarrow$ it never contains variable, but may contain '$\in$'.

  $\rightarrow$ we can always find the first of any variable.

- Follow set $\rightarrow$ Follow set contains terminals and $.

  It can never contain variable and '$\in$'.

  How to find follow set?

1. Include $ in follow of start variable.
2. If production is of type $\rightarrow$

   $A \rightarrow \alpha B \beta$ $\alpha\beta \rightarrow$ strings of grammar symbol].

   follow (B) = first (β).

   If, $\beta \rightarrow \in$, i.e. $A \rightarrow \alpha B$, then follow(B) = follow(A).

- Productions like: $A \rightarrow \alpha A$ gives no follow set.

2.2.6 Example of first and follow set

1. S → AB | CD
   A → aA | a
   B → bB | b
   C → cC | c
   D → dD | d

	First	Follow
S	a, c	$
A	a	b
B	b	$
C	c	D
D	D	$

2.2.7 Entity into Table: Top Down

1. No of Rows = number of unique variable in Grammar.
2. No of columns = [Terminals + $]
3. For a Variable (Row) fill the column (Terminal) if its P, there in its first with the production required.
4. If ∈ is in first put V → ∈ under $ and its follow.

- If any cell has multiple items, then its not possible to have LL (1) Parser, since that will be ambiguous.
- In top down we do: Derivation
  In Bottom up we do: Production.

Question: Construct LL (1) Parsing Table for the given Grammar: E → E + T | T; T → T * F | F; F → (E) | id; ⇒ G_0

⇒ Removing Left Recursion:

E → TE'
E'→ + TE' | ∈
T → FT' G_1
T'→ *FT'| ∈
F → (E) | id

	First	Follow
E	C, id	$,)
E'	+, ∈	$,)
T	C, id	+, $,)
T'	*, ∈	+, $,)
F	C, id	*, +, $,)

- Left Factoring not Required:
  Construction of Table: [LL (1)]

	+	*	(	)	id	$
E	Error	Error	E→TE'	Error	E→ TE'	Error
E'	E'→TE'	Error	Error	E'→∈	Error	E'→∈
T	Error	Error	T→FT'	Error	T→ FT'	error
T'	T'→∈	T'→*FT'	Error	T'→∈	Error	T'→∈
F	Error	Error	F→(∈)	Error	F→ id	error

- Since for G_1, Table constructed with no multiple entries. Hence successfully completed. Hence G_1 is LL (1)

Question: Construct LL (1) Parsing Table for the following Grammar:
S → L = R | R; L → * R | id; R → L ⇒ G_0

Solution: Left Factoring:

$$S \to L = R \mid L \quad L \to *R \mid id \quad R \to L \quad \Rightarrow \quad S \to LX \quad \times \Rightarrow = R \mid \in \quad L \to *R \mid id \quad R \to L \quad \Rightarrow G_1$$

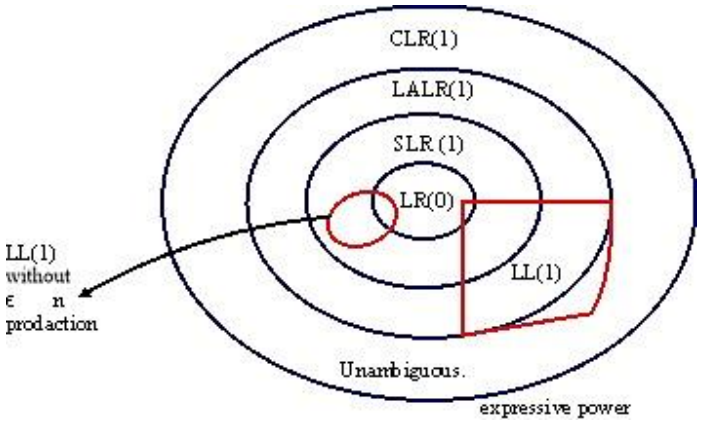
	First	Follow
S	*, id	$
×	=, ∈	$
L	*, id	$
R	*, id	$

Construction of Table:

	*	=	Id	$
S	S→LX	Error	S→L X	Error
L	L→*R	Error	L→id	Error
R	R→L	Error	R→L	Error
×	Error	X→ = R	Error	X→∈

- G_1 is a LL (1) Grammar.

2.2.8 Hierarchy of Parsers: [for ∈- free Grammar]



- For ∈ - producing grammars every LL (1) may not be LALR (1).

Note:
We can't construct any parser for ambiguous grammar. Except: Operator precedence, parser possible for some ambiguous grammar.

Example:

G: S → a S a | b S b | a | b (Unambiguous but no parser) L(G) = ω(a + b) ω R
(Odd palindrome)

- Every RG is not LL (1) as it may be ambiguous, or Recursive or Common Prefix.
- Parsers exists only for the grammar if its Language is DCFL.
- There are some grammars whose Language is DCFL but no parser is possible for it.

2.4.9 Operator Precedence Grammar

Format: (1) No 2 or more variable side by side
(2) No $\in$ production.

Example:

$E \to E + T \mid T$
$T \to T * F \mid F$
$F \to (E) \mid id$
O.G.

or

$E \to E + E$
$E \to E * E$
$E \to a / b$
O.G

or

$S \to aSa \mid bSb \mid a \mid b$
O.G

Or

$S \to AB$
$A \to aA \mid \in$
$B \to bB \mid \in$

Not O.G

2.2.10 Checking LL (1) without Table

$A \to \alpha_1 \mid \alpha_1 \mid \alpha_1$ then $\to$

first $(\alpha_1) \cap$ first $(\alpha_2) = \varnothing$
first $(\alpha_1) \cap$ first $(\alpha_3) = \varnothing$
first $(\alpha_2) \cap$ first $(\alpha_3) = \varnothing$

$A \to \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \in$

first $(\alpha_1) \cap$ first $(\alpha_2) = \varnothing$
first $(\alpha_1) \cap$ first $(\alpha_3) = \varnothing$
first $(\alpha_2) \cap$ first $(\alpha_3) = \varnothing$
follow $(A) \cap$ first $(\alpha_1) = \varnothing$
follow $(A) \cap$ first $(\alpha_2) = \varnothing$
follow $(A) \cap$ first $(\alpha_3) = \varnothing$

2.2.11 Bottom-UP Parser

- It uses RMD in reverse and has no problem with:
  (a) Left recursion (b) Common Prefix
  SLR (1) LR (0) CLR (1) LR (1)
  LR (0) items LALR (1) items

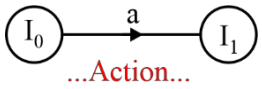
  LR (1) = LR(0) +1 Lookahead
- No parser possible for ambiguous grammar.
- There are some unambiguous grammars for which, there are no Parser.

2.2.12 Basic Algorithm for Construction

- Augment the grammar and expand it, and give numbers to it
- Construct LR (0) or LR (1) item set.
- From that fill the entries in the Table Accordingly.

2.2.13 Types of Entries

(1) Shift Entries: Transitions or Terminals



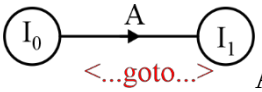
Entry:

	a
I_0	S_1

$a \in$ Terminal I_0, I_1: Item sets.

- Shift entries are some for all Bottom – up Parser.

(2) State Entry: Transition or non – terminal (variable)



$A \rightarrow$ Variable

Entry:

	A
I_0	1

- Same for all Bottom-up Parser.

(3) Reduce Entity: done for each separate production in the item set of type:

$i > \times \rightarrow \alpha$

where, $i \rightarrow$ Production Number

$\times \rightarrow$ Producing Variable

$\alpha \rightarrow$ Grammar String

(a) LR (0) Parser:

Put Ri in every cell of the set-in action table **(ALL).**

(b) SLR (1) Parser:

Put Ri only in the follow(x) form the Grammar. **(Follow(x))**

(c) LALR (1) and CLR (1):

Put Ri only in the look ahead of the production **(Lookaheads)**

2.2.14 Conflicts

LR (0) Parser:

SR: Shift Reduce Conflict

$S \rightarrow \quad X \rightarrow \alpha \cdot t\beta \quad \Rightarrow$ then SR

$R \rightarrow Y \rightarrow \delta \cdot$ conflict

RR: Reduce Reduce conflict

$R \rightarrow X \rightarrow \alpha \cdot \quad \Rightarrow$ then RR

$R \rightarrow X \rightarrow \beta \cdot$

SLR (1) Parser:

SR:

$S \rightarrow X \rightarrow \alpha \cdot t\beta$] $\$ \Rightarrow$then SR

$R \rightarrow Y \rightarrow \delta \cdot$] conflict

$t \in$ follow (Y)]

RR:]

$X \rightarrow \alpha \cdot$] $\$ \Rightarrow$then RR

$Y \rightarrow \beta \cdot$]

Follow (x)n follow (y) $\neq \varnothing$

LALR (1) and CLR (1): Same as SLR (1), but instead

SR:

$X \rightarrow \alpha \cdot t\, \beta \cdot L_1$] $\$ \Rightarrow$then SR

$Y \rightarrow \delta \cdot L_2$] conflict

$T \in L_2$

RR:

$X \rightarrow \alpha \cdot, L_1$] $\$$ then RR

$Y \rightarrow \beta \cdot, L_2$] conflict

$L_1 \cap L_2 \neq \varnothing$

2.4.15 Inadequate Static

A static having ANY conflict is called a conflicting state or independent state.

Note:

The state $S' \rightarrow S.$ or $S' \rightarrow S.,\$$ is accepted state, and this is not a reduction.

- The only difference between CLR (1) and LALR (1) is that, the states with the similar items, but different Lookaheads are merged together to Reduce space.

Important Points

1. If CLR (1) doesn't have any conflict, then conflict may or may not arise after merging in LALR (1).
2. If LALR (1) has SR – conflict, then we can conclude that CLR (1) also has SR – Conflict.
3. LALR (1) has SR – conflict if and only if CLR (1) also has SR.

- We can construct parser for every unambiguous regular grammar: [CLR (1) Parser].

	L Left to right scan of i/p	R Using reverse RMD	(0) No Lookhead	
S Simple	L L to R Scan	R Using Reverse RMD	(1) Lookahead	
L Look	A Ahead	L L to R Scan	R Revers RMD	(1) Lookahead
C Canonical	L L to R Scan	R reverse RMD	(1) Lookahead	

Very Important Point:

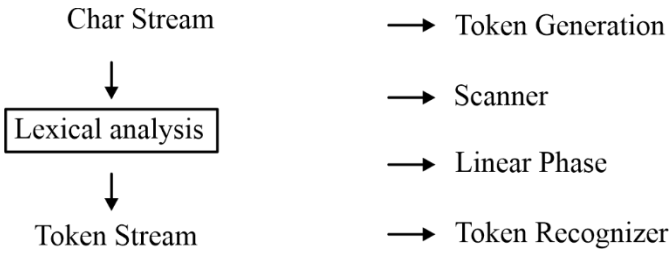
LALR (1) Parser can Parse non LALR (1) grammar, when only non-SR – Conflict by favouring shift over reduce.

Example: $E \rightarrow E + E \mid E * E \mid id \mid 2 + 3 * 5 \Rightarrow E + E \cdot * 5$

❑❑❑

3 LEXICAL ANALYSIS

3.1 Lexical Analysis



3.1.1 Definition

Scan the whole program, char by char and produces the corresponding token.

- Also produces /Lexical Errors (if any).
- Functions of Lexical Analyzer.

(1) Scans all the character of the program.
(2) Token recognizer.
(3) Ignores the comment & spaces.
(4) Maximal Munch Rule [Longest Prefix Match].

Note:
The Lexical Analyser uses, the Regular Expression.
Prioritization of Rules.
Longest Prefix match .

- Lexeme → smallest unit of program or Logic.
- Token → internal representation of Lexeme.

3.1.2 Types of Token

(1) Identifier
(2) Keywords
(3) Operators
(4) Literals/Constants
(5) Special Symbol

3.1.3 Token Separation

(1) Spaces
(2) Punctuation

3.1.4 Implementation

- LEX tool ⇒ Lex.yy.e

- All identifiers will have entry in symbol Table/ LA, gives entries into the symbol Table.
  Regular Expression → DFA → Lexical Analyzer

3.1.5 Find number of Tokens

(1) void main () {
Printf("gate");
}
{11 Tokens}
(2) int x, *P;
x = 10; p = &x; x++;
[18 Tokens]
(3) int x;
x = y;
x == y;
[11 Tokens]
(4) int 1x23; [Lexical Error]
(5) char ch = 'A';
[5 Token]
(6) char ch = 'A;
Lexical Error.
(7) char *p = "gate";
[6 Tokens]
(8) char * p = "gate;
Error.
(9) int x = 10;
/* comment x = x + 1; ERROR
x = x + 1; [11 Tokens]

❑❑❑

4 SYNTAX DIRECTED TRANSACTION

4.1 Syntax Directed Transaction

CFG + Transition
Syntax + Transition
} SDT

SDT: CFG + Transition → 1) Meaning 2) Semantic

4.1.1 Application of SDTL

(1) Used to perform Semantic Analysis.
(2) Produce Parse Tree.
(3) Produce intermediate representation.
(4) Evaluate an expression.
(5) Convert infix to prefix or postfix.

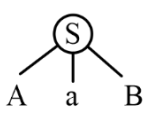
Example: $S \rightarrow S_1S_2$ [S.count = S_1.count + S_2.count]
$S \rightarrow (S_1)$ [S.count = S_1.count + 1]
$S \rightarrow \in$ [S.count = 0]

- Count is an attribute for non–terminal

4.1.2 Attributes

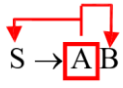
(1) Inherited Attribute
(2) Synthesized Attribute

$S \rightarrow AaB$



S is parent to A, a, B
A is child to S
A is Left Sibling to a, B

(1) Inherited Attribute: (RHS)

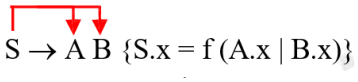


$S \rightarrow AB$ {A.x = f (B.x | S.x)}

- The computation at any node (non-terminal) depends on parent or siblings (S).

- In above Example, x is inherited attribute.

(2) Synthesized Attribute: (LHS)



x is synthesized attribute.

The computation at any node (non-terminal) depends on children.

4.1.3 Identifying Attribute Type

- Always check every Translation.

1) $D \rightarrow T : L$; {L.Type = T.Type} inherited
   $L \rightarrow L1, id$; {L1.Type = L.Type} inherited
   $L \rightarrow id$
   $T \rightarrow integer$ {T.Type = int} synthezised

   Type is neither inherited nor synthesized

2) $E \rightarrow E1 + T$ {E.val = E1.val + T.val} synthesized
   $E \rightarrow T$ {E.val = T.val} Synthesized
   $T \rightarrow T1 - F$ {T.val = T1.val – F.val} Synthesized
   $F \rightarrow id$ {F.val = id.val} synthesized

   Val is synthesized attribute

3) $S \rightarrow AB$ {Aa = B.x; S.y = A.x} x is inherited | y is synthesized
   $A \rightarrow a$ {A.y = a} y is synthesized
   $B \rightarrow b$ {B.y = a.y} y is synthesized

   x ⇒ inherited
   y ⇒ synthesized

4) $D \rightarrow T\ L$ {L.in = T.type} inherited(in)
   $T \rightarrow int$ {T.type = int} /synthesized
   $L \rightarrow id$ {Add type(id.entry,L.in)}

   in ⇒ inherited
   type ⇒ synthesized

4.1.4 Syntax Directed Definitions (SDDs): (Attribute Grammar)

(1) L-Attributed Grammar

→ Attribute is synthesized or restricted inherited. (1) Parent 2) Left sibling only).

→ Translation can be appended anywhere is RHS of production.

→ Example: $S \rightarrow AB$ {A.x = S.x + 2}

or, $S \rightarrow AB$ {B.x = f (A.x | S.x)}

or, $S \rightarrow AB$ {S.x = f (A.x | B.x)}

→ Evaluation: In Order (Topological).

(2) S-Attributed Grammar:

→ Attribute is synthesized only.

→ The transaction is placed only at the end of production.

→ Example: $S \rightarrow AB$ {S.x = f (Ax | Bx)}.

→ **Evaluation:** Reverse RMD (Bottom-Up Parsing).

4.1.5 Identify SDD

(1) $E \rightarrow E_1 + E_2$ {E.type = if (E_1.type = = int & & E.type = = int) then int} Synthesizer else type – error.

$E \rightarrow$ id {E.type = Lookup (id.entry)} synthesizer.

- type is synthesized, hence S-attribute and also L-attributed Grammar.
- Every S-attributed Grammar is also L-attributed Grammar.
- For L-attributed Evaluation, use the In-order of annotated Parser Tree.
- For S-attributed, reverse of RMD is used.

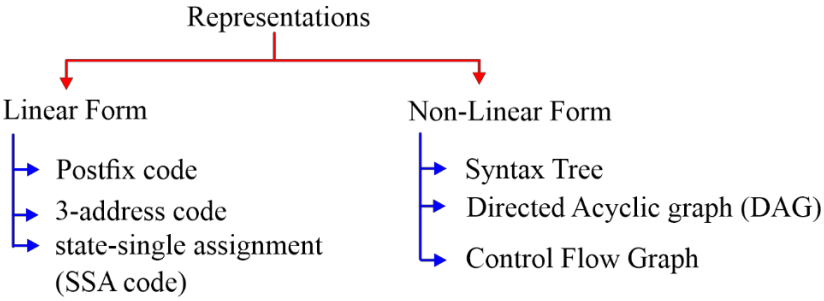
→ find RMD Order.

→ Consider its reverse.

❑❑❑

5 INTERMEDIATE, CODE OPTIMIZATION

5.1 Introduction



Example Expression:

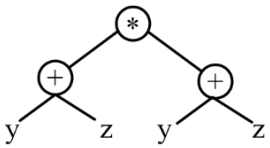
$(y + z) * (y + z)$

Post fix → $\boxed{yz + yz + *}$

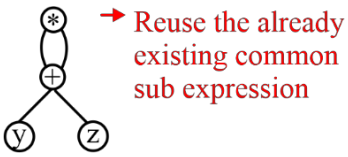
SSA → $\boxed{\begin{aligned} t_1 &= y + z \\ t_2 &= t_1 * t_2 \end{aligned}}$ ⇒f_1 and f_2 cannot be reassigned

3AC → $\boxed{\begin{aligned} t_1 &= y + z \\ t_2 &= t_1 \times t_2 \end{aligned}}$

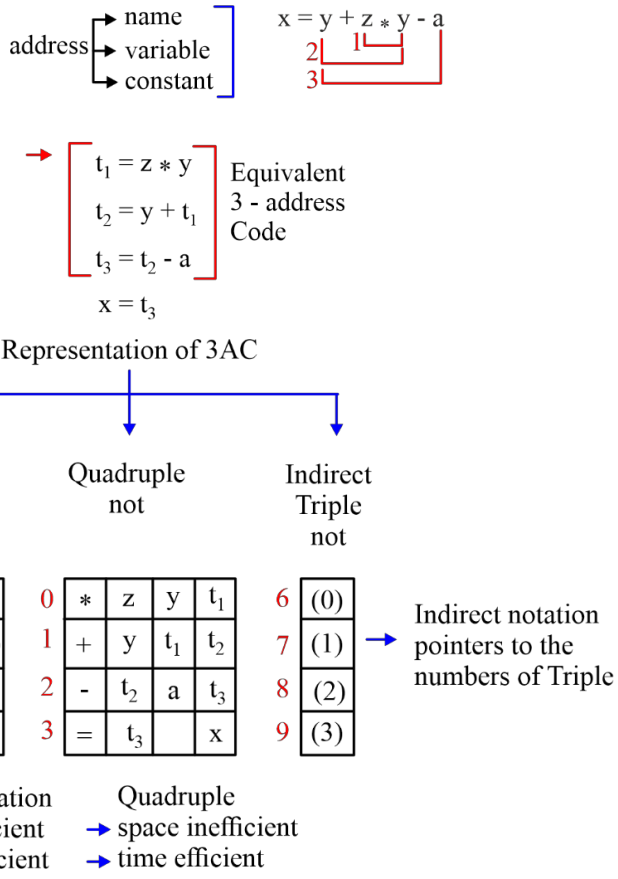
Syntax Tree →



DAG →



3-address Code: Code in which, at most 3 addresses. [including LHS]



- 3AC done using operator precedence.

Find minimum number of variables required in equivalent 3AC:

1.

$x = u - t$ $\qquad x * y \Rightarrow x * (t - z) \Rightarrow (y + w) * (t - z)$

$y = x * v$ $\qquad \Rightarrow ((x * v) + w) * (t - z)$

$x = y + w$ $\qquad \Rightarrow (((u - t) * v) + w) * (t - z)$

$y = t - z$

$y = *x * y$

7 variables

$$\begin{bmatrix} u = u - t & u = t - z \\ v = u * u & z - w * v \\ w = v + w & \end{bmatrix} \Rightarrow 5 \text{ variables only}$$

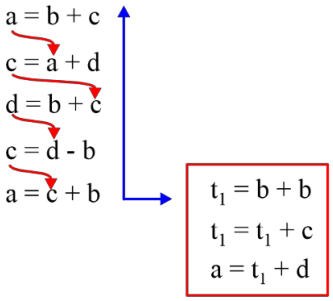
$\Rightarrow$ Evaluating the expression:

a $\Rightarrow c + b \Rightarrow d - b + b \Rightarrow b + c - b + b$

$\Rightarrow b + a + d - b + b$

$\Rightarrow b + b + c + d - b + b$

$\Rightarrow b + b + c + d$



Minimum 3AC.
[wrong, even t_1 is not used]

∴ Minimum:

$$\left\{\begin{matrix} b = b + b \\ c = b + c \\ a = c + d \end{matrix}\right\} \Rightarrow \text{only 3 variables [most optimal]}$$

5.1.1 Static Single Assignment Code: (SSA Code)

Every variable (address) in the code has single assignment [single meaning] + 3AC.

1. x = u - t
   y = x * u
   x = y + w
   y = t - z
   y = x * y

   Find SSA?

⇒ [u, t, v, w, z] are already assigned so we can't use them.

Equivalent SSA Code:

x = u – t y = x * v
p = y + w
q = t – x
r = p * q
in use: x, y, p, q, r ⇒ additional
∴ Total Variable ⇒ 10.

2. p = a - b
   q = p * c
   p = u * v
   q = p + q

⇒ [a, b, c, u, v] are already assigned,

Equivalent SSA Code:

p = a – b
q = p * c
p_1 = v * u
$q_2 = p_1 + q$
in use: p, q, p_1, q_2

∴ Total Variable = 9

5.1.2 Control Flow Graphs

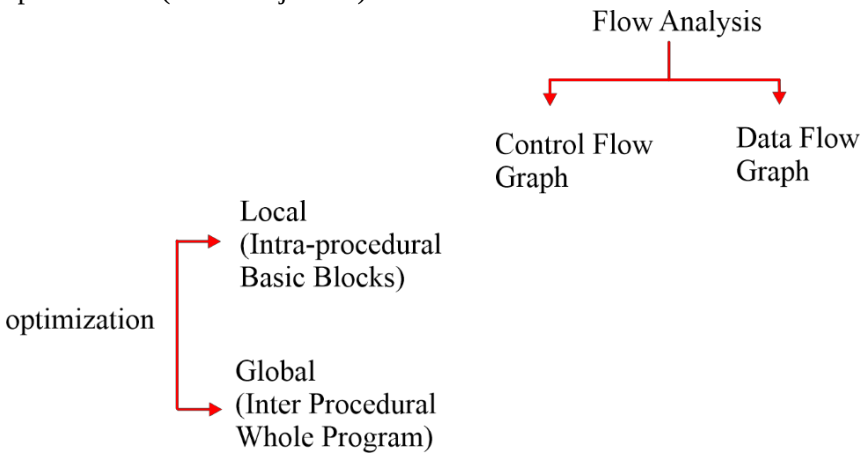
- CFG contain group of basic blocks and controls. CFG has nodes and edges to define basic blocks and controls.
- Basic Blocks: Sequence of 3-address code statements, in which control enters only from 1st statement (called as leader), and leaves from last statement.

Example:

1.	i = 1	LB_1
2.	j = 1	LB_2
3.	t1 = 5 * i	LB_3
4.	t2 = t1 + 5	
5.	t3 = 4*t2	
6.	t4 = t3	
7.	a[t4] = 1	
8.	j = j + 1	
9.	if j ≤ 5 goto 3	LB_4
10.	i = i + 1	LB_5
11.	if i < 5 goto 2	LB_6

5.2 Code Optimization

Saves space / time. (Basic Objective)



5.2.1 Optimization Methods

1. Constant folding
2. Copy propagation
3. Strength Reduction
4. Dead code elimination
5. Common sub expression elimination.
6. Loop Optimization
7. Peephole Optimization

1. Constant Folding

(i) x = 2 * 3 + y ⇒ x = 6 + y

Folding

ii) x = 2 + y * 3} can't fold the constant

2. Copy Propagation

i) Variable Propagation:

x = y;

z = y + 2;

⇒ z = x + 2;

ii) Constant Propagation:

x = 3

z = 3 + a;

⇒ z = x + a

3. Strength Reduction:

Replace expensive statement / instruction with cheaper one.

(i) x = 2 * y **costly** ⇒ x = y + y; **Cheap**

(ii) x = 2 * y ⇒ x = y < < 1; **Much Cheaper**

(iii) x = y / 8 ⇒ x = y > > 3;

4. Dead Code Elimination:

```
x = 2; FALSE                                          x = 2;
if (x > 2)                                            printf("optimization");
        printf ("code"); DEAD CODE CAN BE REMOVED
else
        printf("optimization");
```

- Hence, above dead code never executes during execution. We can always delete such code.

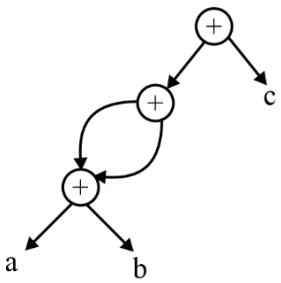
5. Common Sub Expression Elimination:

DAG is used to eliminate common sub expression.

Example: x = (a + b) + (a + b) + c;

⇒ $t_1 = a + b$

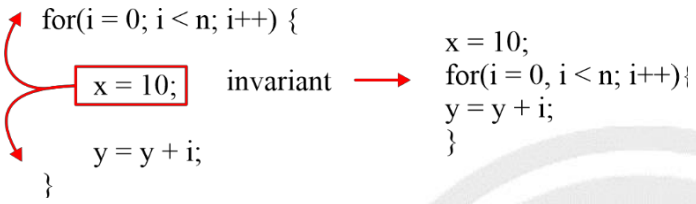
$x = t_1 + t_1 + c$



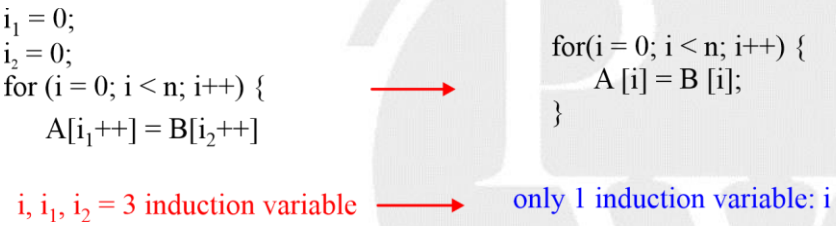
6. Loop Optimization:

(i) Code Motion – Frequency Reduction:

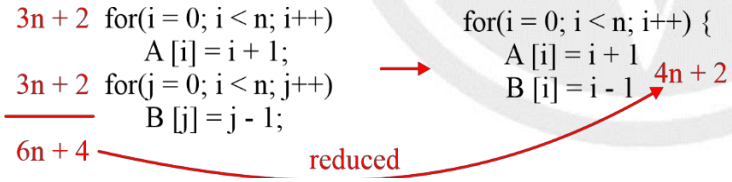
Move the loop invariant code outside of loop.



(ii) Induction Variable elimination:



(iii) Loop Merging / Combining: (Loop Jamming)



(iv) Loop Unrolling:

1)
```
for (i = 0; i < 3; i++)          printf("CD");
    printf("CD");                printf("CD");
                                 printf("CD");
```
3 × 3 + 2 = 11 Statements → 3 statements

2)
```
for (i = 0; i < 2n; i ++) {      for (i = 0; i < n; i++) {
    printf("CD");                    printf("CD");
}                                    printf("CD");
                                 }
```
(2 × 3n + 2) = 6n + 2 → (4n + 2)

7. **Peephole Optimization:**

Examines a short sequence of target instructions in a window (*peephole*) and replaces the instructions by a faster and/or shorter sequence when possible.

- Applied to intermediate code or target code
- Following Optimizations can be used:

- Redundant instruction elimination
- Flow-of-control optimizations
- Algebraic simplifications
- Use of machine idioms

❑❑❑